

**Universidade Federal da Fronteira Sul
Ciência da Computação
Estrutura de Dados II
Professores: Guilherme Dal Bianco**

Niumar Girardi e Leonardo Vitor Roth

**Trabalho TP2
Comparação de Algoritmos de Ordenação**

Repositório Git:
https://github.com/NewtNiu/T2-Estrutura_de_Dados_II-Metodos_de_Ordenacao

Chapecó 2025

Relatório de Desempenho de Algoritmos de Ordenação

1. Introdução

O presente trabalho tem como objetivo **comparar o desempenho de quatro algoritmos clássicos de ordenação** - *Bubble Sort*, *Insertion Sort*, *Heap Sort* e *Quick Sort*.

A análise foi realizada com base em dois critérios principais:

Número total de trocas/operações realizadas durante o processo de ordenação.

Tempo médio de execução, considerando a média de três execuções consecutivas para reduzir o impacto de variações do sistema.

Os testes foram aplicados a vetores de tamanhos **100, 1.000, 5.000, 10.000, 50.000 e 100.000 elementos**, gerados de forma aleatória.

2. Estrutura do Código

O código foi desenvolvido em linguagem C, utilizando uma estrutura modular dividida em arquivos, para facilitar a organização e a manutenção do sistema.

A seguir, apresenta-se uma descrição detalhada de cada parte do código.

2.1. Arquivo main.c

O arquivo principal controla a execução do programa e contém o seguinte fluxo:

- **Geração dos vetores de teste:**
 - Utiliza a função *gerarNumerosAleatorios()* para criar vetores com diferentes tamanhos (100, 1.000, 5.000, 10.000, 50.000 e 100.000 elementos).
- **Menu de seleção:**
 - O usuário escolhe o tamanho do vetor e o algoritmo de ordenação que deseja testar.
- **Criação de cópias:**
 - Cada vetor selecionado é copiado (*copia()*) para garantir que todos os algoritmos recebam a mesma entrada original.
- **Execução e medição:**
 - O tempo de execução é medido utilizando *clock()*, e o número de trocas é armazenado em uma variável do tipo *long long int*.
- **Exibição dos resultados:**
 - Após a ordenação, o programa imprime o nome do algoritmo, o tempo gasto e o número total de trocas/operações.
- **Liberação de memória:**

Vetores dinâmicos são deslocados ao final da execução para evitar vazamentos de memória.

2.2. Arquivo *bubble.c* - Bubble Sort

O algoritmo **Bubble Sort** percorre o vetor várias vezes, trocando elementos adjacentes que estão fora de ordem.

Sua complexidade média e pior caso é **$O(n^2)$** , o que o torna ineficiente para grandes quantidades de dados.

Cada troca é contabilizada no contador de operações.

```
#include "bubble.h"
#include "../utils.h"

void bubbleSort(int *vetor, int tamanho, long long int *contador_trocas) {
    for (int i = 0; i < tamanho - 1; i++) {
        for (int j = 0; j < tamanho - i - 1; j++) {
            if (vetor[j] > vetor[j + 1]) {
                swap(&vetor[j], &vetor[j + 1]);
                (*contador_trocas)++;
            }
        }
    }
}
```

2.3. Arquivo *insertion.c* - Insertion Sort

O **Insertion Sort** insere cada elemento na posição correta do subvetor já ordenado. Também possui complexidade **$O(n^2)$** , mas tende a ter desempenho ligeiramente melhor que o *Bubble Sort* para vetores parcialmente ordenados.

Cada movimentação de elemento é contada como uma operação.

```
#include "../utils.h"
#include "insertion.h"

void insertionSort(int *vetor, int tamanho, long long int *contador_trocas) {
    for (int i = 1; i < tamanho; i++) {
        int chave = vetor[i];
        int j = i - 1;

        while (j >= 0 && vetor[j] > chave) {
            vetor[j + 1] = vetor[j];
            (*contador_trocas)++;
            j--;
        }
        vetor[j + 1] = chave;
    }
}
```

2.4. Arquivo *heap.c* - Heap Sort

O **Heap Sort** utiliza uma estrutura de heap binário para ordenar os elementos.

Sua complexidade é **$O(n \log n)$** em todos os casos.

O algoritmo constrói um *heap máximo*, e em seguida extrai o maior elemento sucessivamente.

Devido à eficiência da estrutura de heap, o número de trocas é significativamente menor.

```
#include "../utils.h"
#include "heap.h"

void heapify(int *vetor, int tamanho, int i, long long int
*contador_trocas) {
    int maior = i;
    int esq = 2 * i + 1;
    int dir = 2 * i + 2;

    if (esq < tamanho && vetor[esq] > vetor[maior])
        maior = esq;

    if (dir < tamanho && vetor[dir] > vetor[maior])
        maior = dir;

    if (maior != i) {
        swap(&vetor[i], &vetor[maior]);
        (*contador_trocas)++;
        heapify(vetor, tamanho, maior, contador_trocas);
    }
}

void heapSort(int *vetor, int tamanho, long long int *contador_trocas)
{
    for (int i = tamanho / 2 - 1; i >= 0; i--) {
        heapify(vetor, tamanho, i, contador_trocas);
    }

    for (int i = tamanho - 1; i > 0; i--) {
        swap(&vetor[0], &vetor[i]);
        (*contador_trocas)++;
        heapify(vetor, i, 0, contador_trocas);
    }
}
```

2.5. Arquivo *quick.c* - Quick Sort

O **Quick Sort** é um dos algoritmos mais eficientes na prática, com complexidade média $O(n \log n)$.

Ele escolhe um pivô e particiona o vetor em duas partes: elementos menores e maiores que o pivô, aplicando recursivamente o mesmo processo.

Mesmo no pior caso (*vetores já ordenados*), pode chegar a $O(n^2)$, mas com boas estratégias de pivô, o desempenho médio é excelente.

```
#include "../utils.h"
#include "quicksort.h"

int partition(int *vetor, int esquerda, int direita,
long long int *contador_trocas) {
    int pivo = vetor[direita];
    int i = esquerda - 1;

    for (int j = esquerda; j < direita; j++) {
        if (vetor[j] <= pivo) {
            i++;
            swap(&vetor[i], &vetor[j]);
            (*contador_trocas)++;
        }
    }
    swap(&vetor[i + 1], &vetor[direita]);
    (*contador_trocas)++;
    return i + 1;
}

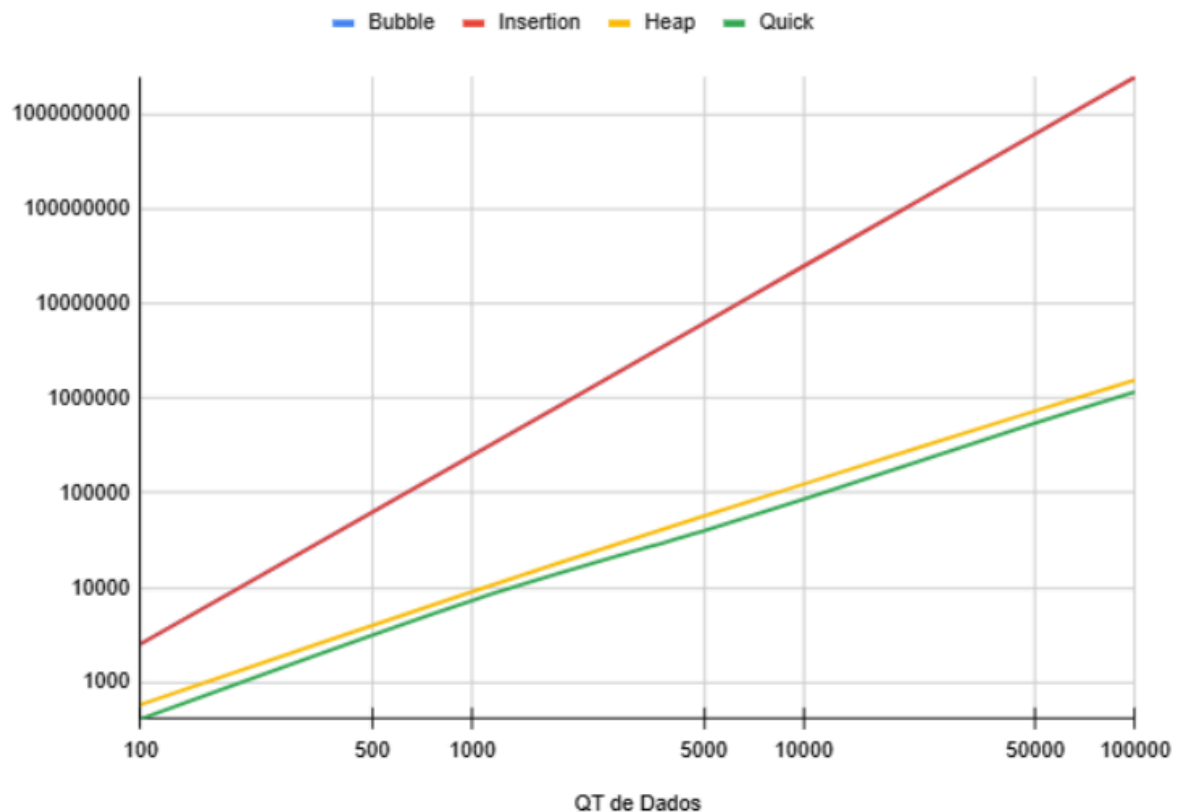
void quickSort(int *vetor, int esquerda, int direita,
long long int *contador_trocas) {
    if (esquerda < direita) {
        int pi = partition(vetor, esquerda, direita, contador_trocas);
        quickSort(vetor, esquerda, pi - 1, contador_trocas);
        quickSort(vetor, pi + 1, direita, contador_trocas);
    }
}
```

3. Resultados e Análise dos Gráficos

Os testes foram realizados com os dados descritos, e as médias foram calculadas a partir de três execuções para cada caso.

Os resultados foram então representados graficamente.

Bubble, Insertion, Heap e Quick



Interpretação:

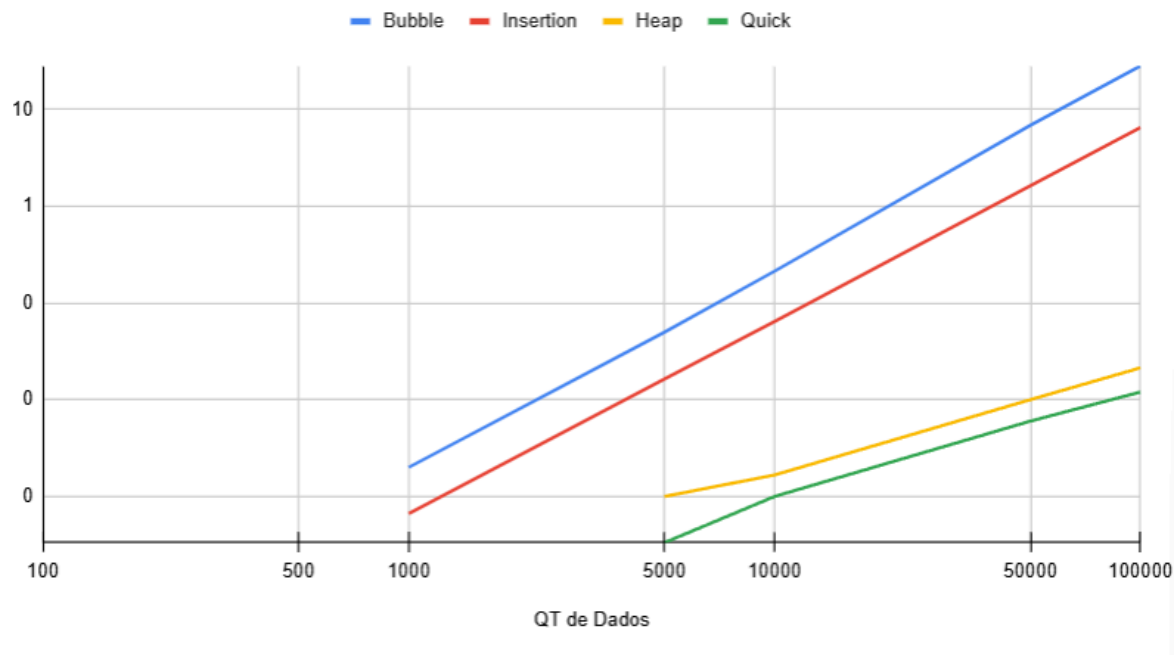
O gráfico mostra o número médio de trocas realizadas por cada algoritmo em diferentes tamanhos de vetores.

QT de Dados	Bubble	Insertion	Heap	Quick
100	2534	2534	582	409
1000	249397	249397	9086	7318
5000	6223223	6223223	57163	39846
10000	24980812	24980812	124256	86231
50000	625957318	625957318	737430	550344
100000	2501341350	2501341350	1574779	1167861

Análise:

- Bubble Sort e Insertion Sort apresentaram número idêntico de trocas, ambos com crescimento quadrático.
- Heap Sort realizou muito menos trocas, mantendo-se praticamente linear em escala logarítmica.
- Quick Sort foi o mais eficiente, com o menor número de operações em todos os casos testados.

Tempo de Execução



Interpretação:

O gráfico apresenta o tempo médio de execução (em segundos) de cada algoritmo conforme o tamanho dos vetores aumenta.

QT de Dados	Bubble	Insertion	Heap	Quick
100	0	0	0	0
1000	0,002	0,000666666667	0	0
5000	0,04966666667	0,016333333333	0,001	0,000333333333
10000	0	0,064333333333	0,001666666667	0,001
50000	6,819666667	1,621666667	0,01	0,006
100000	28,097	6,494666667	0,021333333333	0,012

Análise:

- O Bubble Sort teve o maior tempo de execução, crescendo exponencialmente conforme o tamanho do vetor.
- O Insertion Sort apresentou leve melhora, mas ainda é ineficiente para grandes volumes.
- O Heap Sort foi muito rápido, com tempos na ordem de milissegundos.
- O Quick Sort se destacou como o mais veloz, com o menor tempo médio em todos os testes.

4. Conclusão

A partir dos resultados obtidos, é possível observar diferenças significativas de desempenho entre os quatro algoritmos analisados.

Número de Trocas / Operações

- Bubble Sort e Insertion Sort apresentaram praticamente o mesmo número de trocas em todos os tamanhos de vetores.
- Ambos possuem complexidade $O(n^2)$, o que os torna ineficientes para grandes quantidades de dados.
- Heap Sort realizou muito menos trocas, demonstrando maior eficiência estrutural.
- Quick Sort apresentou o menor número de operações, confirmando sua eficiência prática.

Tempo de Execução

- Bubble Sort: ~28 segundos para 100.000 elementos.
- Insertion Sort: ~6,5 segundos para 100.000 elementos.
- Heap Sort: ~0,02 segundos.
- Quick Sort: ~0,012 segundos.

Considerações Finais

- Para **vetores pequenos**, as diferenças são discretas, mas com o crescimento dos dados, *Quick Sort* e *Heap Sort* se destacam fortemente.
- *Bubble Sort* e *Insertion Sort* são úteis apenas em **contextos didáticos**, para demonstrar o funcionamento de algoritmos de ordenação simples.
- O *Quick Sort* obteve o **melhor desempenho geral**, confirmando sua complexidade média $O(n \log n)$ e eficiência prática.

Esses resultados reforçam a importância da **escolha adequada do algoritmo de ordenação**, que impacta diretamente na velocidade e no custo computacional de um sistema.